

TITLE: EXP NO.: 6 IMPLEMENTING FABRIC CHANNELS FOR PRIVACY

1. AIM

To create and configure multiple channels in a Hyperledger Fabric network, deploy chaincode on a specific channel, and demonstrate how Fabric channels provide data privacy and isolation between organizations.

2. TOOLS / SOFTWARE REQUIRED

- Kali Linux / Ubuntu
- Docker
- Docker Compose
- Node.js (Version 18 or later)
- npm
- Git
- Hyperledger Fabric Samples
- Hyperledger Fabric Binaries
- Hyperledger Fabric Docker Images
- Visual Studio Code (Optional)

3. HARDWARE REQUIREMENTS

- Processor: Intel Core i3 or above
- RAM: Minimum 8 GB
- Storage: Minimum 20 GB Free Disk Space
- Internet Connection

4. THEORY

A Fabric Channel is a private communication mechanism that allows a subset of organizations to maintain a separate ledger and execute transactions confidentially. Each channel has its own blockchain ledger, smart contracts (chaincode), and world state database. Organizations that are not members of a channel cannot access its ledger or transaction data. Channels enable enterprise applications to maintain confidentiality while sharing the same Hyperledger Fabric network.

5. PROCEDURE

Step 1: Navigate to the Fabric Test Network

```
cd ~/fabric-dev/fabric-samples/test-network
```

Step 2: Start the Hyperledger Fabric Network

```
./network.sh up createChannel -ca
```

Step 3: Verify the Existing Channel

List the channels joined by the peer.

```
peer channel list
```

Step 4: Create a New Private Channel

Create a second channel named privatechannel.

```
./network.sh createChannel -c privatechannel
```

Step 5: Verify Available Channels

```
peer channel list
```

Step 6: Deploy Chaincode on the Private Channel

Deploy the JavaScript Asset Transfer chaincode only to privatechannel.

```
./network.sh deployCC  
-c privatechannel  
-ccn basic  
-ccl javascript  
-ccp ../asset-transfer-basic/chaincode-javascript
```

Step 7: Initialize the Ledger

Initialize the ledger for the privatechannel.

```
peer chaincode invoke  
-o localhost:7050  
--ordererTLSHostnameOverride orderer.example.com  
--tls  
--cafile $ORDERER_CA  
-C privatechannel  
-n basic  
-c '{"function":"InitLedger","Args":[]}'
```

Step 8: Query Assets from the Private Channel

```
peer chaincode query  
-C privatechannel  
-n basic  
-c '{"Args":["GetAllAssets"]}'
```

Step 9: Verify Channel Isolation

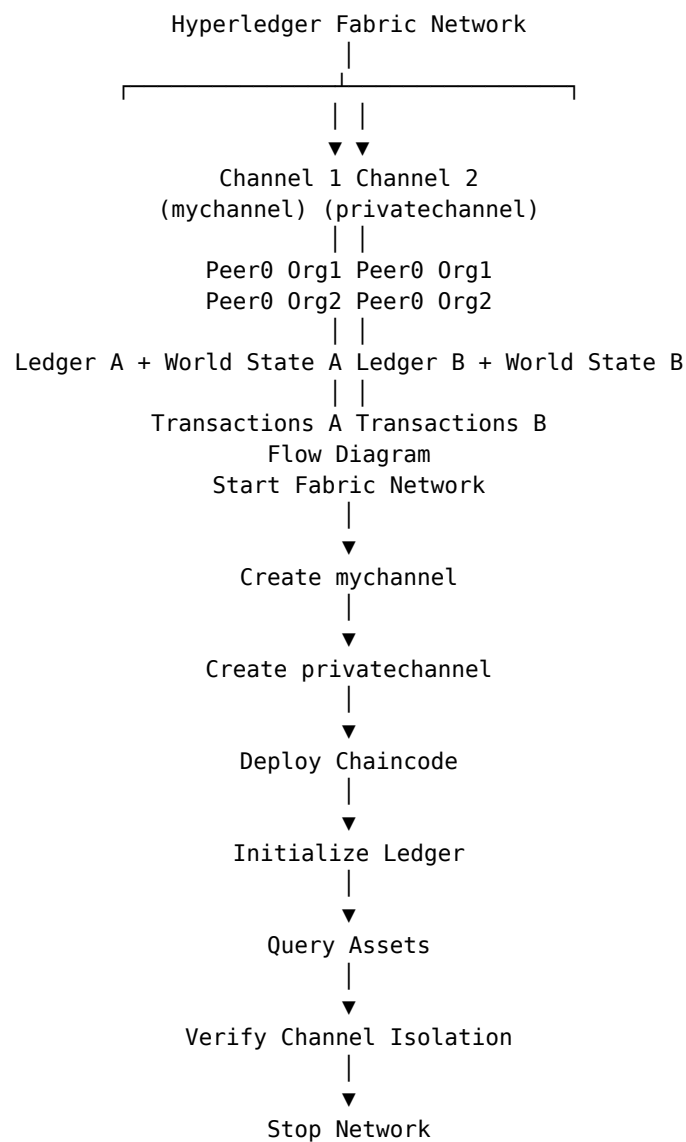
Attempt to query privatechannel data from mychannel.

```
peer chaincode query
-C mychannel
-n basic
-c '{"Args":["ReadAsset","asset1"]}'
```

Step 10: Stop the Fabric Network

```
./network.sh down
```

6. ARCHITECTURE DIAGRAM



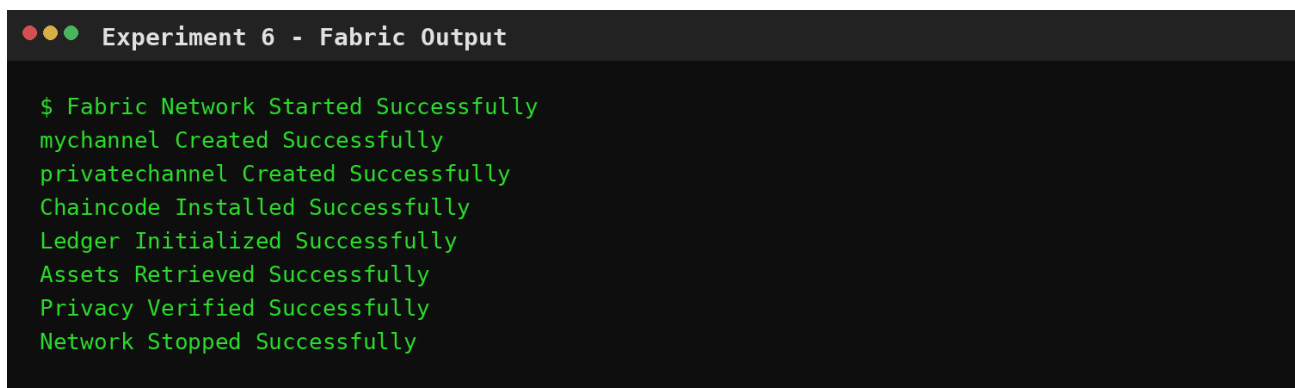
7. SUMMARY OF KEY COMMANDS

Purpose	Command
Navigate	<code>cd ~/fabric-dev/fabric-samples/test-network</code>
Start network	<code>./network.sh up createChannel -ca</code>
Verify channel	<code>peer channel list</code>
Command	<code>./network.sh createChannel -c privatechannel</code>
Deploy chaincode	<code>./network.sh deployCC</code>
Invoke transaction	<code>peer chaincode invoke</code>
Query ledger	<code>peer chaincode query</code>
Stop network	<code>./network.sh down</code>

8. OUTPUT

The terminal output below is rendered from the output blocks supplied in the source experiment.

Output 1



```

Experiment 6 - Fabric Output

$ Fabric Network Started Successfully
mychannel Created Successfully
privatechannel Created Successfully
Chaincode Installed Successfully
Ledger Initialized Successfully
Assets Retrieved Successfully
Privacy Verified Successfully
Network Stopped Successfully

```

Fig. 6.1 – Fabric output corresponding to the source experiment output.

9. RESULT

The Hyperledger Fabric network was successfully configured with multiple channels. A private channel was created, chaincode was deployed, and ledger transactions were executed successfully. The experiment demonstrated that channel members can access only the data belonging to their respective channels, ensuring privacy and confidentiality in a permissioned blockchain network